ASİNXRON Və SİNXRON PROQRAMLAŞDIRMAYA GİRİŞ

Synchronous (Sinxron)(blocking)

- Tapşırıqlar ardıcıllıqla icra olunur.
- Yəni biri bitmədən digəri başlamır.

- Misal

Siz marketin kassasında sıradasınız kassir önünüzdəki müştərinin yola salmadan sizə xidmət edə bilmir.

```
Console.WriteLine("1. Fayl oxumağa başlayıram...");
string data = File.ReadAllText("bigfile.txt"); // Bloklayır
Console.WriteLine("2. Fayl oxundu!");
```

```
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        Console.WriteLine("Start");

        // Bloklayan sinkron iş
        DoWork();

        Console.WriteLine("End");
    }

    static void DoWork()
    {
        Console.WriteLine("Work started...");
        Thread.Sleep(3000); // 3 saniyə bloklayır
        Console.WriteLine("Work finished!");
    }
}
```

Asynchronous (Asinxron)(Nonblocking)

•Tapşırıq başladıqdan sonra **onu gözləmədən** digər işlərə keçmək mümkündür.
•Cavab hazır olanda xəbər gəlir (callback, event, future, promise və s.).

•**Misal:** Restorana sifariş verirsiniz. Ofisiant sifarişi mətbəxə ötürür və digər müştərilərə də xidmət

edir. Yemək hazır olanda sizə gətirir.

```
Console.WriteLine("1. Fayl oxumağa başlayıram...");
string data = await File.ReadAllTextAsync("bigfile.txt"); // Asinxron
Console.WriteLine("2. Fayl oxundu!");
```

```
static async Task Main()
{
    Console.WriteLine("Start: " + DateTime.Now);

    // 3 async iş
    Task<string> httpTask = FetchHttpAsync("https://jsonplaceholder.typicode.com/todos/1");
    Task<string> fileTask = ReadFileAsync("test.txt");
    Task<string> dbTask = SimulateDbQueryAsync();

    Console.WriteLine("Main thread is free to do other work: " + DateTime.Now);

    // Hamısını paralel gözləyirik
    var results = await Task.WhenAll(httpTask, fileTask, dbTask);

    Console.WriteLine("All tasks finished: " + DateTime.Now);

    Console.WriteLine("HTTP Result: " + results[0]);
    Console.WriteLine("File Result: " + results[1]);
    Console.WriteLine("DB Result: " + results[2]);
}
```

```
static async Task<string> FetchHttpAsync(string url)
{
    Console.WriteLine("HTTP request started: " + DateTime.Now);
    using HttpClient client = new HttpClient();
    string result = await client.GetStringAsync(url); // Non-blocking
    Console.WriteLine("HTTP request finished: " + DateTime.Now);
    return result;
}

1 reference
static async Task<string> ReadFileAsync(string filePath)
{
    Console.WriteLine("File read started: " + DateTime.Now);

    if (!File.Exists(filePath))
    {
        await File.WriteAllTextAsync(filePath, "Hello async file!");
    }

    string content = await File.ReadAllTextAsync(filePath); // Non-blocking
    Console.WriteLine("File read finished: " + DateTime.Now);
    return content;
}

1 reference
static async Task<string> SimulateDbQueryAsync()
{
    Console.WriteLine("DB query started: " + DateTime.Now);
    await Task.Delay(2000); // Simulate non-blocking DB query
    Console.WriteLine("DB query finished: " + DateTime.Now);
    return "DB result";
}
```

- Task.Delay(3000) non-blockingdir.Main Thread i bloklamır.Digər işləri görə bilir.
- Əgər Thread.Sleep(3000) istifadə etsəydik,Main Thread 3 saniyə bloklanardı.

Paralel proqramlaşdırma eyni vaxtda bir neçə hesablama vahidində iş aparmaqdır.

- CPU/GPU resurslarını tam istifadə edib işi sürətləndirmək,
- Böyük həcmli verilənləri qısa zamanda emal etmək

CONCURRENCY VS PARALLELISM

•**Concurrency (eyni vaxtlılıq)** — bir neçə tapşırığın "idarə edilməsi" (interleaving) — eyni anda tək bir nüvədə də mümkün (time-slicing).
•**Parallelism (paralellik)** — eyni işin eyni anda faktiki olaraq bir neçə nüvədə icrası.
Praktik olaraq qarışdırılır, amma dizayn və problemlər müxtəlif olur (məs. resurs bölüşdürmə vs sinxronizasiya).

1. CONCURRENCY

- Birdən çox iş eyni anda idarə olunur,amma mütləq olaraq eyni anda icra olunmur.
- Bu bir CPU nüvəsində də mümkünkür.CPU kiçik zaman dilimlərinə bölünür və hər tapşırığa ardıcıl olaraq bir az vaxt verilir.İnsan gözü üçün eyni vaxtda işləyir kimi görünür.

- Misal:

1 aşpazın eyni anda bir neçə yeməyi bişirməsi concurrency'dir. Misal: aşpaz əsas yeməyi bişməsi üçün sobaya verdikdən sonra, digər yemək üçün kartof soymağa başlayır, sobadakı yemək bişdikdən sonra onu serviz edir və soymuş olduğu kartofu sobaya göndərib ardınca isə salat üçün göyərti doğramağa başlayır. Burada diqqət edilməsi gərəkən yer eyni anda sadəcə 1 işin görüldüyüdür. Aşpazın 4 əli olmadığı üçün eyni anda həm kartof soyub həm göyərti doğraya bilmir.

PARALLELISM (EHTIYATLI FƏRQLƏNDIRMƏ)

- **Parallelism (paralellik)** — eyni işin eyni anda faktiki olaraq bir neçə nüvədə icrası. Praktik olaraq qarışdırılır, amma dizayn və problemlər müxtəlif olur (məs. resurs bölüşdürmə vs sinxronizasiya).

- Parallelizmdə isə işin daha sürətli görülməsi üçün 1'dən çox aşpaz (thread) daxilir edilir mətbəxə (proqrama). 3 aşpaz - biri əsas yeməyi, biri qarniri digəri isə salatı hazırlayır.

CPU Bound

- CPU ya bağlıdır.Əsas işi CPU da hesablama aparmaqla keçir.Yəni hansısa prosesin hansı sürətlə yerinə yetirilməsi CPU nun gücünə bağlıdır.
- CPU zəifdirsə yavaş işləyəcək.

Niyə “tezləşdirmək” olmur?

- Çünki burada gözləmə yoxdur(diskdən oxuma,internetdən cavab,DB sorğusu)
- Amma bəzi hallarda alqoritmi optimallaşdırmaq da sürət verə bilər(məsələn O(n^2) yerinə

O(n logn) istifadə etmək).

I/O-BOUND

- I/O(Input/Output)

Diskdən fayl oxuma/yazma

Şəbəkdən məlumat alma(API)

Database sorğusu işlətmək

Klaviaturadan input oxuma

- İşin əsas vaxtı gözləməyə gedir-CPU çox az işləyir,amma çox gözləyir.

#Input(daxil etmə) – proqramın hər hansı bir xarici mənbədən məlumat alması

Output(xaric etmə) – proqramın xarici mühitdə məlumat göndərməsi

Thread Nədir?

- Əgər biz normalda 2 method yaradıb onu run etsək görərik ki 1ci method işini bitirikdən sonra 2ci method işə düşür. Lakin *Thread*- Paralel bir şey icra etmək istədikdə istifadə edilir. Threadlər CPU üçün müəyyən bir kod blokudur .C# 7.0 in a Nutshell kitabında Threading bölməsindəki bir kod nümunəsini yazmaq istəyirəm. Thread yaratmaq üçün Thread classından istifadə edilib və bu da hər hansısa bir parametr(WriteY() methodunu) qəbul edir. Threadi trigger(oyatmaq) etmək üçün “Start()” methodunu mütləq yazmalıyıq. Digər “for” dövri isə *main thread* tərəfindən içra olunur. Biz bu kod blokunu dəfələrlə run etsək görərik ki hər dəfə fərqli cavablar alırıq. Buna da CPU özü qərar verir ki hansı threaddən başlasın və nə qədər icra olunduqdan sonra digər threadə kecsin.

- *Thread bir işi parçalara bölüb hər parçanı ayrılıqda və hamısını eyni zamanda etmək imkanıdır. Bu birbaşa komputerin CPU -su ilə bağlı bir işdir. Yəniki biz bir işi hissələrə bölüb prosessora göndəririk, prosessorda hərəsini bir nüvədə işləyib bizə nəticəni daha tez qaytarır (prosessorun imkanlarından aslı olaraq bəzən bir nüvədə də paralel işləməsi mümkündür). Yaxudda bir iş eyni anda bir neçə bölmədə icra edilir.Main Thread hər C# proqramı başladığı anda avtomatik bir əsas thread yaradır.Sənin Main() metodu və ya Program.cs içindəki kod Thread-də icra olunur.*

TASK VƏ TASK<T>
NECƏ THREAD ÜZƏRİNDƏ İŞLƏYİR

- Task - .NET-də asinxron və paralel əməliyyatları idarə etmək üçün yaradılmış yüksək səviyyəli abstraksiyadır.
- Yəni, əslində **Thread-lər üzərində işləyir**, amma onları **öz içində idarə edir** və bizə **daha rahat və təhlükəsiz API** təqdim edir.
- Task arxa planda **Thread Pool**-dan istifadə edir.

Thread Pool Nədir?

- Thread Pool - .Net in yaratdığı Thread lar anbarıdır.
- Əgər hər dəfə yeni Thread açsan,sistem resursları çox istifadə olunur.
- Amma Task isə mövcud olan boş thread-i Thread Pool dan götürür,sonra istifadə edir,sonra geri qoyur.
- Daha az RAM və CPU istifadə olunur.
- Tasklar tez başlayır və tez tamamlanır.
- Threadlərin idarəsi .Net tərəfindən idarə olunur.

- #Thread əməliyyat sistemi səviyyəsində icra xəttidir, Task isə onun üzərində .NET abstraction-dır və ThreadPool istifadə edir.

```
using System;
using System.Threading.Tasks;

class Program
{
    static void Main()
    {
        Task.Run(() =>
        {
            Console.WriteLine($"Task Thread ID: {Thread.CurrentThread.ManagedThreadId}");
        });

        Console.WriteLine("Main bitdi!");
        Console.ReadLine();
    }
}
```

Task.Run() əslində yeni Thread açmır.Thread Pool-dan birini götürür.

Task<t> Nədir?

- Nəticə qaytaran Task dır.

```
Task<int> t = Task.Run(() =>
{
    Console.WriteLine("Hesablama gedir...");
    Thread.Sleep(2000);
    return 42;
});

Console.WriteLine($"Nəticə: {t.Result}");
```

Async Və Await

- Async – metodun asinxron işləyəcəyini bildirir
- Await – Task-in bitməsini gözləyir,amma thread – i bloklamır.

Əvvəllər biz Thread,Task.Wait() və Task.Result ilə işləyəndə:

- Thread bloklanırdı.
- UİI və ya Main thread donurdu.
- Resurs israfı olurdu.

- “Məlumat gələnə qədər mən gedim başqa iş görüm.
  Məlumat gələndə xəbər et, qayıdım davam edim.”

Thread Bloklanmaq Nəyə Deyilir?

- “Məlumat gələnə qədər mən başqa heç nə etməyəcəm”
- Nəticədə bu thread dayanırdı, başqa iş görə bilmirdi.
  Buna **bloklanma (blocking)** deyilir.

Paralel Proqramlaşdırma(#CPU İŞLƏRI ÜÇÜN)

Əvvəl öyrəndiyimiz async / await → *eyni anda bir neçə işmiş kimi görünsə də*, əslində: bir thread işləyir, amma ağıllı şəkildə "vaxtı bölür".
Amma Parallel Programming → *həqiqətən bir neçə thread eyni anda işləyir*.

```
Parallel.For(1, 6, i =>
{
    Console.WriteLine($"İş {i} → Thread ID: {Thread.CurrentThread.ManagedThreadId}");
});
Console.WriteLine("Bütün işlər tamamlandı!");
```

```
Microsoft Visual Studio Debu

Is 3 ? Thread ID: 5
Is 1 ? Thread ID: 9
Is 5 ? Thread ID: 11
Is 2 ? Thread ID: 12
Is 4 ? Thread ID: 13
Butun isl?r tamamland?!

C:\Users\Acer08\Desktop\ConsoleApp1\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 25512) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Niyə Praktikada “Asinxronluq” Daha Çox Işlənir?

- Çünki **real dünyadakı proqramların əksəriyyəti I/O-bound** olur.

Məsələn API yazırsan

- Istifadəçi gəlir login olur
- Şəkil yükləyir
- Məlumat oxuyur

Bu işlərin hamısı **I/O əməliyyatlarıdır** — yəni CPU çox iş görmür, sadəcə nəticəni gözləyir.
Əgər sən bu zaman **Thread-i bloklasan**, o sadəcə boş-boşuna gözləyəcək.

- Amma “Parallel” hər zaman faydalı deyil
- Hər işi **fiziki thread**-də paralel edir.
- Thread-lər çox resurs tələb edir (RAM, scheduling vaxtı).
- Əgər çox Parallel.For açsan ,CPU overload ola bilər.

- Yəni asinxronluq – CPU ya boş dayanma digər iş gör
- Paralellik CPU ya hər nüvə ilə eyni anda çox iş gör deyir.

State machine-proqramın hansı mərhələdə olduğunu izləyən sistemdir.

- Async metodu çağıranda C# compiler (tərtibçi) avtomatik olaraq sənin kodunu **state machine** şəklinə çevirir.

Əslində async metodu **daxildə bir neçə vəziyyətə (state)** bölünür:

- State 0 → başlamamış
- State 1 → await nöqtəsinə qədər gəldi
- State 2 → await sonrası icra olunur
- State -1 → tamamlandı

State	Vəziyyət	Hərəkət
-1	Bitmiş	Task tamamlandı
0	Başlanğıc	await-dən əvvəlki kod işləyir
1	Gözləmə	await nəticə verməyib, gözləyir
2	Davam	await bitdi, sonrası icra olunur

```
╔═══════════════════════╗
‖ Task (işin özü)      ‖ ← vəziyyəti: Running, Completed, Faulted
╚═══════════════════════╝

        |
        ▼

╔═══════════════════════╗
‖ State Machine        ‖ ← vəziyyəti: await-dən əvvəl, await zamanı, await-dən sonra
╚═══════════════════════╝
```

ValueTask nədir?

- ValueTask Task üçün daha yeni və daha yüngül alternativdir.
- O, nəticənin dərhal əldə edilə biləcəyi kritik performans ssenariləri üçün nəzərdə tutulmuşdur (məsələn, keşdən).

```
async Task<int> GetNumberAsync()
{
    return 42;
}
```

- Yəni sən hər dəfə bu metod çağrılanda Task<int> obyekt olduğu üçün heap də yaradılır.İçində 42 dəyəri saxlanılır.
- ValueTaskın üstünlüyü odur ki bu bir də obyekt yaradıb heapdə yer tutmur.Dəyər stackdə saxlanılır.

Parallel class nədir?

- Parallel - .NET in System.Threading.Tasks namespace-də olan bir sinifidir.
- Məqsədi birdən çox işi eyni anda fərqli threadlərdə icra etməkdir.

- Normal For və ya ForEach ardıcıl işləyir.
- Amma Parallel.For və Parallel.ForEach eyni anda çox iş görür.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"İş {i} - Thread: {Thread.CurrentThread.ManagedThreadId}");
    Thread.Sleep(1000); // İş 1 saniyə çəkir
}
```

```
Is 0 - Thread: 1
Is 1 - Thread: 1
Is 2 - Thread: 1
Is 3 - Thread: 1
Is 4 - Thread: 1

C:\Users\Acer08\Desktop\ConsoleApp1\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 3444) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

ParallelLoopResult nə edir?

- Parallel.For və Parallel.ForEach geri ParallelLoopResult qaytarır.

```
ParallelLoopResult result = Parallel.For(0, 5, i =>
{
    Console.WriteLine($"Task {i}");
});
```

```
Task 2
Task 3
Task 4
Task 0
Task 1
```

Parallel.Invoke — birdən çox metodu paralel işlətmək

```
Parallel.Invoke(
    () => Task1(),
    () => Task2(),
    () => Task3());


void Task1() => Console.WriteLine($"Task1 - {Thread.CurrentThread.ManagedThreadId}");
void Task2() => Console.WriteLine($"Task2 - {Thread.CurrentThread.ManagedThreadId}");
void Task3() => Console.WriteLine($"Task3 - {Thread.CurrentThread.ManagedThreadId}");
```

```
Task2 - 9
Task3 - 10
Task1 - 1

C:\Users\Acer08\Desktop\ConsoleApp1\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 28060) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

```
Parallel.For(0, 5, i =>
{
    Console.WriteLine($"İş {i} - Thread: {Thread.CurrentThread.ManagedThreadId}");
    Thread.Sleep(1000); // İş 1 saniyə çəkir
});
```

```
Is 1 - Thread: 12
Is 4 - Thread: 7
Is 3 - Thread: 9
Is 0 - Thread: 11
Is 2 - Thread: 10

C:\Users\Acer08\Desktop\ConsoleApp1\ConsoleApp1\bin\Debug\net8.0\ConsoleApp1.exe (process 31148) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .
```

Race condition nədir?

- **Race condition** — bir neçə thread eyni **paylaşılan resursa (məsələn, dəyişənə) eyni anda giriş edib onu dəyişdiyi** zaman yaranan **nəticə qeyri-sabitliyi**dir.

```
int count = 0;

Parallel.For(0, 1000, i =>
{
    count++;
});

Console.WriteLine(count);
```

- Yəni 2 thread eyni anda işə düşsə eyni nəticəni göstərəcək.Ardıcıl işləsə fərqli çünki 1 ci thread 2 ci thread i görür.

- Həlli:lock

- “Bu kod parçasına eyni anda **yalnız bir thread** daxil ola bilər. Qalanları kənarda **gözləməlidir**.”

Thread `Monitor.Enter(obj)` çağırır.

CLR obyektin header-də baxır: “Bu obyektin monitoru kimdədi?”

- Əgər boşdursa → Thread onu tutur (lock alır).
- Əgər artıq başqa thread tutubsa → Thread “waiting queue”-ya düşür.

Monitor “ownership” saxlayır — hansı thread-in tutduğunu bilir.

`lock` blokundan çıxanda `Monitor.Exit(obj)` çağırılır və növbəti thread buraxılır.

Mutex nədir?

- Lock və monitor eyni proqram daxilində işləyir.
- Yəni 2 Thread eyni proqram daxilindədirsə -lock işləyəcək.
- Amma 2 ayrı proqram eyni resursa(məsələn,eyni fayla,eyni DB ə) çıxış etməyə çalışarsa lock artıq işə yaramayacaq,çünki onlar ayrı proqramdırlar və bir-birlərinin lock obyektini görmürlər.
- Burada mutex köməyə gəlir.
- Mutex (mutual exclusion) o deməkdir ki, “eyni anda yalnız bir proses və ya thread daxil ola bilər”.

App1

```csharp
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        // Adlı Mutex yaradırıq
        using (Mutex mutex = new Mutex(false, "Global\\MyAppMutex"))
        {
            Console.WriteLine("App1 Mutex gözləyir...");
            mutex.WaitOne(); // Mutex-i alırıq

            Console.WriteLine("App1 daxil oldu.");
            Thread.Sleep(5000); // İş görür (5 saniyəlik)
            Console.WriteLine("App1 çıxır.");

            mutex.ReleaseMutex(); // Mutex-i buraxırıq
        }
    }
}
```

App2

```csharp
using System;
using System.Threading;

class Program
{
    static void Main()
    {
        using (Mutex mutex = new Mutex(false, "Global\\MyAppMutex"))
        {
            Console.WriteLine("App2 Mutex gözləyir...");
            mutex.WaitOne(); // App1 buraxana qədər gözləyəcək

            Console.WriteLine("App2 daxil oldu.");
            Thread.Sleep(5000);
            Console.WriteLine("App2 çıxır.");

            mutex.ReleaseMutex();
        }
    }
}
```

Əgər App1 ilk açılarsa,Mutex-i götürəcək və App2 gözləyəcək.
App1 Mutex i buraxanda,App2 davam edəcək.
Bu fərqli proqramlar arasında sinxronizasiya deməkdir.

Monitor

- Lock arxasında işləyən sinifdir.
- Əslində C# -da lock sözü syntactic sugardır.

csharp Copy code

```csharp
lock (obj)
{
    // critical section
}
```

slində **compiler** tərəfindən belə çevrilir 👇

csharp Copy code

```csharp
Monitor.Enter(obj);
try
{
    // critical section
}
finally
{
    Monitor.Exit(obj);
}
```

Interlocked

Interlocked bu problemi **atomik** əməliyyatlarla həll edir.
Atomik (atomic) əməliyyat nədir?
O deməkdir ki, əməliyyat ya tamamilə icra olunur,ya da heç icra olunmur,Yəni əməliyyat bölünməzdir-bir addımda baş verir.

```
public  int Say=0;
2 references
public void Counter()
{
    Interlocked.Increment(ref Say);
}
```

```
Program program = new Program();
Thread thread1 = new Thread(program.Counter);
thread1.Start();
Thread thread2 = new Thread(program.Counter);
thread2.Start();


thread1.Join();
thread2.Join();

Console.WriteLine($"Say: {program.Say}");
```

Nəticə 2 olacaq burda çünki 1ci Thread bizə 1 nəticəsini 2ci Thread isə 1 i görür və üzərinə 1 gələrək 2 nəticəsini bizə qaytarır.İnterlocked atomik əməliyyat icra etdiyinə görə 1 addım həll edir.Əks halda məsələn Say++ istifadə etsək bu zaman 3 addımda həll edəcək yəni 1ci oxuyacaq sonra 1 əlavə edəcək sonra nəticəni qaytaracaq bu zaman əgər 2 Thread eyni anda işə düşdüyü zaman 1 ci Thread 0 oxuyur və 2 ci Thread də 0 oxuyacaq və nəticə olaraq 1 qayıdacaq.

İ/O bound da cpu niyə işləmir?

```
using System;
using System.Diagnostics;
using System.IO;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        var sw = Stopwatch.StartNew();

        // I/O-bound — disk əməliyyatı
        await File.ReadAllTextAsync("test.txt"); // faylı oxuyur

        sw.Stop();
        Console.WriteLine($"I/O-bound əməliyyat vaxtı: {sw.ElapsedMilliseconds} ms");
    }
}
```

🎈 Burada:

- Əsas iş **disk driver** tərəfindən aparılır, CPU sadəcə sorğunu göndərir.
- CPU fayl oxunarkən **boş dayanmaq yerinə başqa iş görə bilər.**
- Ona görə biz `await` istifadə edirik ki, CPU bloklanmasın.